

Afyra Mkononi — Product Requirements Document

DRAFT V1.0

OWNER: NAOMI CHERONO

LAST UPDATED: 2026-05-17

"AI-powered patient support for smarter clinic workflows." Afya Mkononi (Swahili: "health in hand") is a 24/7 AI healthcare assistant enabling patients to access general health guidance, book appointments, and set reminders — while automatically escalating emergencies to professional care. This PRD is the scope contract for the four-week MVP build.

The Problem

Small and mid-sized clinics field the same routine inquiries daily — hours, rescheduling, symptom checks, medication timing. These essential-but-repetitive queries consume disproportionate clinical time. Patients, meanwhile, delay care due to uncertainty and miss appointments due to poor reminders. The result: avoidable workload, slower care, and under-triaged emergencies.

The Solution

A 24/7 digital channel that handles the routine layer — general guidance, appointment booking, and reminder capture — while escalating anything emergent. Clinical staff reclaim time for interactions that require human judgment. Every session, booking, and reminder is persisted in PostgreSQL for clinic-side visibility.

Target Users

Patients (Primary)

- Adults seeking general health information
- People managing ongoing care — medication schedules, follow-ups
- People uncertain whether symptoms warrant a visit

Clinic Staff (Secondary)

- Clinical officers and nurses fielding repetitive questions
- Clinic administrators managing appointment books
- Small private clinic owners seeking lightweight digital tools

Out of Scope (v1)

- Children (no pediatric guidance)
- Complex chronic care coordination
- Insurance / billing workflows
- Emergency-only users

MVP Scope — What's In v1

- 1

Public Home Page
Explains the product with three primary CTAs: Chat, Book, Remind.
- 2

Patient Chatbot
Conversational AI for general health questions with persisted session history and safety boundaries.
- 3

Appointment Booking
Patient submits name, contact, preferred date/time, and reason. Admin sees PENDING rows.
- 4

Reminder Creation
Medication, appointment, or follow-up reminders captured; delivery deferred to v2.
- 5

Django Admin
Clinic-side view of all appointments, reminders, and chat sessions with filters and search.
- 6

Deployed Live Demo
Accessible at a public URL; all data persisted in PostgreSQL.

⚠️ Out of Scope for v1 — Check Every Feature Request Here First: Medical diagnosis · Prescription generation · Emergency handling beyond escalation · Insurance/payments · EMR integration · Patient record integration · User authentication · SMS/email reminder delivery · Multilingual UI · Native mobile apps · Role-based access control

Key Functional Requirements

Chatbot

- Anonymous sessions (no sign-in required); UUID persisted server-side
- Every message persisted with a safety_category: NORMAL, EMERGENCY, REFUSED, ESCALATED
- Emergency keywords trigger explicit "seek urgent medical care immediately" response
- Visible safety disclaimer above input field at all times
- 5 suggested prompt chips; loading indicator on AI reply
- AI must never diagnose or recommend specific medications

Appointments

- Required: patient name, phone, preferred date/time, reason
- Phone validation; inline errors preserve form state
- Submission creates Appointment with status=PENDING; confirmation screen shows ID
- Admin: filterable by status and date; status updatable (PENDING → CONFIRMED / CANCELLED / COMPLETED)

Reminders

- Types: Medication / Appointment / Follow-up / Other
- Stored with status=PENDING; SMS/email delivery is v2
- Admin: filterable by type, status, and date

Admin

- Superuser views all appointments, reminders, and chat sessions
- Chat sessions display messages inline, ordered chronologically
- Search and filters on status, date, patient name, phone

Non-Functional Requirements

-  **Performance**

Pages render <2s on 3G · AI responses arrive <5s · Loading state during wait
-  **Security**

No secrets in source · CSRF protection · Rate limit 30 req/min/IP · Content sanitized · PII not logged
-  **Reliability**

Graceful AI failure messages · All DB writes in transactions · No orphan rows
-  **Accessibility**

Labeled form elements · WCAG AA contrast · Keyboard-navigable chat
-  **Usability**

Works from 375px width · All three core flows completable in <60s · Disclaimer always in viewport
-  **Maintainability**

AI calls isolated to ai_service.py · Apps split by responsibility · System prompt as code constant

Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
AI produces unsafe medical advice despite system prompt	Medium	High	Emergency keyword detection before LLM sees message; system prompt enforces refusal; safety_category persisted for audit
Provider rate limits / cost overruns during dev	Medium	Medium	Use cheapest model (Claude Haiku / GPT-4o-mini); hard daily spend cap; log every call
Scope creep into v2 features	High	High	Section 7 (Out of Scope) is the canonical reference for every "should we add X?" question
Postgres setup pain on Windows	Medium	Low	Docker Postgres fallback documented in implementation-plan.md
Safety regression in late-stage UI changes	Low	High	Emergency-keyword tests written Day 1 of Week 3 AI work, not Day 7

Build Success Metrics

- Live URL accessible from any browser
- A stranger completes all three core flows without help
- All 7 emergency keywords reliably produce escalation messaging
- Django admin shows real data from real user submissions
- All 13 Definition-of-Done items in implementation-plan.md checked

v2 Backlog (Parked, Not Lost)

01	02
User authentication Patient and clinic-admin login; tie records to accounts	SMS & email reminder delivery Africa's Talking / Twilio integration
03	04
Multilingual support Swahili first, then expand	Better triage flows Structured decision trees layered on AI chat
05	06
Custom clinic admin dashboard Metrics, filters, bulk actions beyond Django admin	Native mobile apps & voice interface iOS + Android; voice for low-literacy users